ĐẶC TẢ THIẾT KẾ gemm_main_fsm

Mục lục

1 Tổng quan **2**
1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 3

2 Đặc tả thanh ghi **8**

3 Mô tả chức năng **8**
3.1 Tổ chức chức năng . . . 8
3.2 Dựng và kiểm tra job . . . 8
3.3 Bank và cấp quyền bắt đầu pha . . . 9
3.4 Engine . . . 10
3.5 Điều khiển tác vụ . . . 12
3.6 Timing . . . 13
3.7 Reset, hiện thực, kiểm chứng . . . 14

1. Tổng quan

1.1. Tính năng chính

Main FSM là module duy nhất ra lệnh cho DMA và array. Mỗi phép GEMM được biểu diễn bằng một **job** (lấy từ bản chụp thanh ghi ở chế độ đơn, hoặc từ một descriptor ở chế độ chuỗi). Ba engine độc lập đưa job qua các pha: **Đọc** (FETCH, kiểm tra, LOAD_*), **Tính** (COMPUTE) và **Ghi** (STORE_C). Bộ chấm điểm bank quyết định khi nào một pha được bắt đầu, nên khi có ping-pong ba pha của ba job liên tiếp chạy chồng nhau mà không bao giờ đụng cùng một bank.

Chức năng

- Chế độ đơn (START): một job, các pha CHECK_CFG → LOAD_BIAS (nếu `POST_EN`) → LOAD_A → LOAD_B → COMPUTE → STORE_C, tuần tự
- Chế độ chuỗi (START_CHAIN): mỗi descriptor là một job, chạy FETCH → kiểm tra → LOAD_BIAS → LOAD_A → LOAD_B (trừ khi `REUSE_B`) → COMPUTE → STORE_C ; chuỗi dừng ở descriptor có `LAST` = 1
- Với `HAS_PINGPONG` = 1, ba engine có thể chạy đồng thời khi các điều kiện bank và thứ tự job bên dưới cho phép. Với `HAS_PINGPONG` = 0, mọi pha dùng bus hoặc scratchpad loại trừ nhau.
- Nội dung DRAM sau một tác vụ DONE giống hệt thực thi tuần tự từng job, với mọi cấu hình tham số và mọi thời gian trả lời của bus
- FSM báo kết thúc tác vụ cho Register File bằng đúng một xung done hoặc một xung error; điều phối soft reset; cấp bộ đếm `DESC_DONE_CNT`, thanh ghi `ERR_DESC_ADDR` và tín hiệu đếm hiệu năng
- FSM cấp cấu hình của job đang tính cho array và VPU, và bank đang tính cho scratchpad. Bản chụp của Register File chỉ đi tới FSM

1.2. Tham số

Bảng 1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Kích thước array; 4 hoặc 8. Chỉ dùng cho `HW_CONFIG` và assertion
`HAS_CHAIN`	1	1: có chế độ chuỗi. 0: START_CHAIN kết thúc với mã lỗi 5 ; engine Đọc không có FETCH
`HAS_PINGPONG`	1	1: hai bank mỗi buffer, các pha chạy chồng. Đòi hỏi `HAS_CHAIN` = 1
`NB`	2	localparam = `HAS_PINGPONG` ? 2 : 1, số bank mỗi buffer

Bảng 2: Hành vi theo cấu hình

Cấu hình	Hành vi
`HAS_CHAIN` = 0, `HAS_PINGPONG` = 0	Chế độ đơn. Mọi job có bank 0; chỉ một job mỗi tác vụ
`HAS_CHAIN` = 1, `HAS_PINGPONG` = 0	Chuỗi descriptor, các pha tuần tự tuyệt đối (quy tắc SEQ, mục 3.3.3)
`HAS_CHAIN` = 1, `HAS_PINGPONG` = 1	Bản đầy đủ: chuỗi descriptor, ping-pong
`HAS_CHAIN` = 0, `HAS_PINGPONG` = 1	Không hợp lệ: chặn lúc elaboration

1.3. Tín hiệu giao tiếp

Bảng 3: Tín hiệu giao tiếp của `gemm_main_fsm`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Clock, reset, soft reset			
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`o_soft_clr`	1	Output	Xung xóa đồng bộ một chu kỳ gửi đến các module. Chỉ phát sau khi cả hai kênh DMA đã rảnh; hủy trạng thái tác vụ hiện tại.
`o_soft_busy`	1	Output	Báo đang xử lý soft reset. Trong khoảng này Register File báo BUSY = 1 và không nhận tác vụ mới.
Register File → FSM			
`i_start_pulse`	1	Input	Xung một chu kỳ báo Register File vừa chấp nhận một tác vụ. Loại lệnh và bản chụp cấu hình hợp lệ cùng chu kỳ.
`i_start_kind`	2	Input	Loại lệnh được chấp nhận. Bit 0 ứng với START; bit 1 ứng với START_CHAIN. Giá trị 01: tác vụ đơn; 10: chuỗi; 11: cấu hình lệnh sai.
`i_soft_req`	1	Input	Xung một chu kỳ yêu cầu Main FSM thực hiện soft reset. Phát khi lệnh ghi CTRL có SOFT_RESET = 1 và byte 0 được cho phép ghi.
`i_cfg_dim`	32	Input	Bản chụp đủ 32 bit của MATRIX_DIM. Bit [7:0]: M; [15:8]: N; [23:16]: K; [31:24]: dự trữ. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_a_addr`	32	Input	Bản chụp đủ 32 bit của BASE_ADDR_IN. Địa chỉ byte của ma trận A. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_b_addr`	32	Input	Bản chụp đủ 32 bit của BASE_ADDR_WEIGHT. Địa chỉ byte của ma trận B. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_c_addr`	32	Input	Bản chụp đủ 32 bit của BASE_ADDR_OUT. Địa chỉ byte vùng kết quả C hoặc Y. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_post_ctrl`	32	Input	Bản chụp đủ 32 bit của POST_CTRL. Bit 0 bật hậu xử lý; bit 1 bật ReLU. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_bias_addr`	32	Input	Bản chụp đủ 32 bit của BIAS_ADDR. Địa chỉ byte của vector bias. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.

Bảng 3: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_cfg_qm0`	32	Input	Bản chụp đủ 32 bit của QUANT_M0. Hệ số nhân lượng tử M0, INT32 có dấu. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_qshift`	32	Input	Bản chụp đủ 32 bit của QUANT_SHIFT. Bit [4:0] là lượng dịch sh; giữ cả các bit cao để kiểm tra cấu hình. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`i_cfg_desc_ptr`	32	Input	Bản chụp đủ 32 bit của DESC_PTR. Địa chỉ descriptor đầu chuỗi; bằng 0 khi không hỗ trợ chuỗi. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
FSM → Register File			
`o_done_pulse`	1	Output	Xung một chu kỳ báo toàn bộ tác vụ kết thúc thành công. Register File đặt DONE và xóa BUSY tại cạnh lên; không xuất hiện cùng err pulse.
`o_err_pulse`	1	Output	Xung một chu kỳ báo toàn bộ tác vụ kết thúc với lỗi. Mã lỗi hợp lệ cùng chu kỳ; Register File đặt ERROR và xóa BUSY tại cạnh lên.
`o_err_code`	4	Output	Mã lỗi của tác vụ, được lấy mẫu cùng err pulse. Giá trị 1: cấu hình. Giá trị 2: đọc dữ liệu. Giá trị 3: ghi kết quả; 4: đọc descriptor; 5: tính năng không có.
`o_perf_load`	1	Output	Báo pha LOAD dữ liệu, không bao gồm FETCH đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`o_perf_comp`	1	Output	Báo pha COMPUTE đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`o_perf_store`	1	Output	Báo pha STORE_C đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`o_err_desc_addr`	32	Output	Địa chỉ descriptor lỗi được cung cấp cho thanh ghi ERR_DESC_ADDR tại offset 0x44. Bằng 0 ở chế độ đơn hoặc khi không hỗ trợ chuỗi.
`o_desc_done_cnt`	32	Output	Số descriptor đã hoàn tất STORE_C thành công, cung cấp cho thanh ghi DESC_DONE_CNT tại offset 0x48. Bằng 0 ở chế độ đơn hoặc khi không hỗ trợ chuỗi.
Lệnh kênh đọc DMA			
`o_rd_start`	1	Output	Xung một chu kỳ yêu cầu bắt đầu lệnh đọc DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có o_rd_start = 1.

Bảng 3: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_rd_dst`	2	Output	Chọn đích của lệnh đọc. Giá trị 0: buffer A. Giá trị 1: buffer B. Giá trị 2: buffer BIAS. Giá trị 3: descriptor trả về Main FSM. Giá trị được chốt cùng xung o_rd_start.
`o_rd_base`	32	Output	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh đọc DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung o_rd_start.
`o_rd_rows`	6	Output	Số hàng cần truyền trong lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_rd_start.
`o_rd_rwords`	6	Output	Số word 32 bit cần truyền trong mỗi hàng của lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_rd_start.
`o_rd_stride`	16	Output	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung o_rd_start.
`o_rd_bank`	1	Output	Chọn bank đích của buffer A, B hoặc BIAS khi nạp dữ liệu; không dùng cho FETCH. Giá trị được chốt cùng xung o_rd_start.
`o_rd_abort`	1	Output	Yêu cầu dừng lệnh đọc DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`i_rd_busy`	1	Input	Báo kênh đọc DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận o_rd_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`i_rd_done`	1	Input	Xung một chu kỳ báo lệnh đọc DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`i_rd_error`	1	Input	Xung một chu kỳ báo lệnh đọc DRAM kết thúc có lỗi. Lỗi gồm phản hồi R khác OKAY hoặc RLAST sai vị trí. Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
`i_desc_valid`	1	Input	Báo có một word descriptor hợp lệ trong chu kỳ này. Chỉ dùng với lệnh FETCH; bên nhận lấy dữ liệu khi valid = 1, không có ready.
`i_desc_idx`	4	Input	Chỉ số word trong descriptor, từ 0 đến 15. Hợp lệ cùng chu kỳ với i_desc_valid.
`i_desc_data`	32	Input	Nội dung 32 bit của word descriptor có chỉ số i_desc_idx. Hợp lệ cùng chu kỳ với i_desc_valid; dữ liệu của FETCH bị lỗi không được dùng để thực thi job.

Bảng 3: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Lệnh kênh ghi DMA			
`o_wr_start`	1	Output	Xung một chu kỳ yêu cầu bắt đầu lệnh ghi DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có o_wr_start = 1.
`o_wr_fmt`	1	Output	Chọn định dạng dữ liệu trong buffer C. Giá trị 0: INT32, mỗi phần tử chiếm một word. Giá trị 1: INT8, bốn phần tử trong một word. Giá trị được chốt cùng xung o_wr_start.
`o_wr_base`	32	Output	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh ghi DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung o_wr_start.
`o_wr_rows`	6	Output	Số hàng cần truyền trong lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_wr_start.
`o_wr_rwords`	6	Output	Số word 32 bit cần truyền trong mỗi hàng của lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_wr_start.
`o_wr_stride`	16	Output	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung o_wr_start.
`o_wr_bank`	1	Output	Chọn bank nguồn của buffer C khi thực hiện STORE_C. Giá trị được chốt cùng xung o_wr_start.
`o_wr_abort`	1	Output	Yêu cầu dừng lệnh ghi DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`i_wr_busy`	1	Input	Báo kênh ghi DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận o_wr_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`i_wr_done`	1	Input	Xung một chu kỳ báo lệnh ghi DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`i_wr_error`	1	Input	Xung một chu kỳ báo lệnh ghi DRAM kết thúc có lỗi. Lỗi xảy ra khi phản hồi B khác OKAY. Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
Array, VPU và scratchpad phía tính			
`o_pe_start`	1	Output	Xung một chu kỳ bắt đầu COMPUTE cho toàn bộ job. Chỉ phát khi array rảnh; kích thước và cấu hình tính toán phải hợp lệ cùng chu kỳ.
`i_pe_done`	1	Input	Xung một chu kỳ báo COMPUTE đã hoàn tất, bao gồm hậu xử lý và ghi kết quả vào buffer C. Không báo cho lượt tính bị soft clear hủy.

Bảng 3: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_pe_busy`	1	Input	Báo array đang tính. Bằng 1 từ chu kỳ sau khi nhận pe start đến hết chu kỳ có pe done; về 0 khi soft clear hủy lượt tính.
`o_comp_dim`	32	Output	Kích thước job. Bit [7:0]: M; [15:8]: N; [23:16]: K. M từ 1 đến 32; N và K là bội của 4, từ 4 đến 32. Phải hợp lệ khi bắt đầu lượt tính. Bit [31:24] bằng 0; giữ nguyên từ pe start đến hết pe done.
`o_comp_post_en`	1	Output	Cho phép hậu xử lý: 0 xuất INT32 nguyên gốc; 1 cộng bias, lượng tử hóa và xuất INT8. Giữ ổn định từ pe start đến hết pe done.
`o_comp_relu_en`	1	Output	Cho phép chặn kết quả âm về 0 trước khi kẹp INT8. Bị bỏ qua khi hậu xử lý tắt. Giữ ổn định từ pe start đến hết pe done.
`o_comp_qm0`	32	Output	Hệ số nhân lượng tử M0, số nguyên 32 bit có dấu. Giữ ổn định từ pe start đến hết pe done.
`o_comp_qshift`	5	Output	Lượng dịch bổ sung sh, số không dấu từ 0 đến 31. Tổng lượng dịch trong công thức lượng tử là 31 + sh. Giữ ổn định từ pe start đến hết pe done.
`o_comp_bank_a`	1	Output	Chọn bank của buffer A dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`o_comp_bank_b`	1	Output	Chọn bank của buffer B dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`o_comp_bank_bias`	1	Output	Chọn bank của buffer BIAS dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`o_comp_bank_c`	1	Output	Chọn bank của buffer C dành cho phía tính. Giữ nguyên trong suốt COMPUTE; khi chỉ có một bank thì dùng bank 0.
`o_comp_active`	1	Output	Báo COMPUTE đang hoạt động. SRAM dùng tín hiệu này để kiểm tra quyền truy cập bằng assertion; tín hiệu không thay thế các enable đọc/ghi.

Giao tiếp

- Mọi port `o_*` lấy từ thanh ghi, trừ `o_rd_abort` và `o_wr_abort` (mức, cũng từ thanh ghi trạng thái)
- Lệnh DMA: FSM chỉ bật `o_rd_start` khi kênh đọc rảnh (`i_rd_busy` = 0 và không có lệnh nào đang chờ `i_rd_done`/`i_rd_error`); tương tự kênh ghi. Các port tham số chỉ cần hợp lệ ở chu kỳ `o_*_start` = 1; DMA tự chốt
- `o_comp_dim`, `o_comp_post_en`, `o_comp_relu_en`, `o_comp_qm0`, `o_comp_qshift`, `o_comp_bank_*` hợp lệ từ chu kỳ `o_pe_start` = 1 tới hết chu kỳ `i_pe_done` = 1 và không đổi trong khoảng đó
- `o_pe_start` chỉ khi `i_pe_busy` = 0. Mỗi `o_pe_start` có đúng một `i_pe_done`
- `o_done_pulse` và `o_err_pulse` không bao giờ cùng chu kỳ; mỗi `i_start_pulse` có tối đa một trong hai; không có cái nào cho tác vụ bị soft reset hủy

2. Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Cấu hình được nhận qua các tín hiệu đầu vào; trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3. Mô tả chức năng

3.1. Tổ chức chức năng

Bảng 4: Module con

Module	Instance	Vai trò
`gemm_fsm_ctrl`	`u_ctrl`	Trạng thái tác vụ; xử lý `i_start_kind`; bản ghi lỗi; tín hiệu chặn; cấp quyền bắt đầu pha; done/error; soft reset; `ERR_DESC_ADDR`
`gemm_job_check`	`u_chk` (trong engine Đọc)	Tổ hợp: dựng job, kiểm kích thước, địa chỉ, cờ, bit dự trữ và stride theo các điều kiện trong mục Dựng và kiểm tra job; tính stride hiệu dụng.
`gemm_rd_engine`	`u_rd`	Engine Đọc: FETCH, CHECK, chờ WAIT_PREV, LOAD_*, đẩy job vào slot
`gemm_cp_engine`	`u_cp`	Engine Tính: lấy job từ slot, chạy COMPUTE, giữ job tới khi engine Ghi nhận
`gemm_wr_engine`	`u_wr`	Engine Ghi: STORE_C; `DESC_DONE_CNT`; báo job cuối
`gemm_bank_sb`	`u_sb`	Chọn bank theo con trỏ của từng buffer; bộ đếm chiếm dụng bank

3.1.1. Job

Một job là struct `gemm_pkg::job_t`. Engine Đọc giữ một job, slot giữ một job, engine Tính và engine Ghi mỗi bên giữ một job; tối đa bốn job cùng tồn tại.

Bảng 5: Trường của `job_t`

Trường	Rộng	Nội dung
`idx`	32	Số thứ tự trong chuỗi, 0, 1, 2, …; luôn 0 ở chế độ đơn
`daddr`	32	Địa chỉ descriptor (cho `ERR_DESC_ADDR`); 0 ở chế độ đơn
`m, n, k`	6×3	M, N, K (1 … 32)
`a, b, c, bias`	32×4	Địa chỉ byte của A, B, C/Y, bias
`lda, ldb, ldc`	16×3	Stride hiệu dụng (byte), đã thay 0 bằng độ dài hàng
`qm0, qsh`	32, 5	M_0, sh
`post, relu, reuse, waitp, last`	1×5	Cờ
`next`	32	`NEXT_PTR` (chỉ engine Đọc dùng)
`ba, bb, bbias, bc`	1×4	Bank của A, B, BIAS, C (gán ở CHECK)

3.2. Dựng và kiểm tra job

3.2.1. Chế độ đơn

Bảng 6: Job từ bản chụp (`gemm_job_check`, mode = đơn)

Trường	Giá trị
`m, n, k`	`i_cfg_dim[7:0]`, `[15:8]`, `[23:16]` (6 bit thấp)
`a, b, c, bias`	`i_cfg_a_addr`, `i_cfg_b_addr`, `i_cfg_c_addr`, `i_cfg_bias_addr`
`lda, ldb, ldc`	K; N; N nếu `post`, ngược lại 4N
`post, relu`	`i_cfg_post_ctrl[0]`, `[1]`

Bảng 6: Job từ bản chụp (`gemm_job_check`, mode = đơn) (tiếp theo)

Trường	Giá trị
`qm0`, `qsh`	`i_cfg_qm0`, `i_cfg_qshift[4:0]`
`idx`, `daddr`, `reuse`, `waitp`, `last`	0, 0, 0, 0, 1

Job hợp lệ khi $1 \leq$ M $\leq$ 32; N, K thuộc 4, 8, …, 32; bit [31:24] của kích thước bằng 0; các địa chỉ A, B, C căn 4 byte; POST_CTRL[31:2] = 0; RELU_EN chỉ được bật cùng POST_EN. Khi POST_EN = 1, địa chỉ bias căn 4 byte và QUANT_SHIFT[31:5] = 0. Khi POST_EN = 0, bỏ qua bias và tham số lượng tử. Phải kiểm tra bản chụp đủ 32 bit trước khi rút gọn trường.

3.2.2. Chế độ chuỗi

Engine Đọc nhận 16 word descriptor qua `i_desc_*` và lưu word 0 … 10 vào bộ đệm. Word 11 (`TAG`) được bỏ qua; word 12 … 15 chỉ cần OR dồn để kiểm bằng 0.

Bảng 7: Job từ descriptor

Trường	Giá trị (wn = word thứ n)
`m`, `n`, `k`	w0[7:0], w0[15:8], w0[23:16]
`a`, `b`, `c`	w1, w2, w3
`lda`, `ldb`	w4[15:0], w4[31:16]; 0 $\rightarrow$ K, N
`ldc`	w5[15:0]; 0 $\rightarrow$ N nếu `post`, ngược lại 4N
`last`, `waitp`, `reuse`, `post`, `relu`	w6[0], w6[1], w6[2], w6[3], w6[4]
`next`, `bias`, `qm0`, `qsh`	w7, w8, w9, w10[4:0]
`idx`, `daddr`	Bộ đếm descriptor của engine Đọc; con trỏ dùng để FETCH

Kiểm tra descriptor (`gemm_job_check`, mode = chuỗi)

- Hợp lệ khi và chỉ khi mọi điều sau đúng : $1 \leq M \leq 32$; N, K $\in$ {4, 8, …, 32}; w0[31:24] = 0; w5[31:16] = 0; w6[31:5] = 0; w10[31:5] = 0; w12 … w15 = 0
- `a`[1:0] = 0 và `c`[1:0] = 0; `b`[1:0] = 0 nếu `reuse` = 0; `bias`[1:0] = 0 nếu `post` = 1
- Mỗi stride thô (w4[15:0], w4[31:16], w5[15:0]) bằng 0, hoặc là bội của 4 và không nhỏ hơn độ dài hàng: K cho A; N cho B; 4N cho C khi `post` = 0, N khi `post` = 1
- `relu` = 1 kéo theo `post` = 1
- `reuse` = 1 chỉ khi `idx` > 0 và (K, N) bằng (K, N) của job trước trong chuỗi (thanh ghi `prev_kn` của engine Đọc)
- `last` = 0 kéo theo `next`[5:0] = 0 . Vi phạm được tính là lỗi của chính descriptor này
- `DESC_PTR`[5:0] $\neq$ 0 được kiểm ở `gemm_fsm_ctrl` khi bắt đầu chuỗi: lỗi mã 1 với f = 0, `ERR_DESC_ADDR` = `DESC_PTR`, không truy cập bus

3.3. Bank và cấp quyền bắt đầu pha

3.3.1. Chọn bank

`gemm_bank_sb`: chọn bank

- Thanh ghi con trỏ `nxt_a`, `nxt_c`, `nxt_bias`, `nxt_b`, `prev_b`: về 0 khi chấp nhận tác vụ, khi `rst_n` và khi `soft_clr`
- Ở chu kỳ CHECK của job hợp lệ: `ba=nxt_a`, `bc=nxt_c`; `bbias=nxt_bias` nếu `post` (ngược lại 0); `bb=prev_b` nếu `reuse`, ngược lại `bb=nxt_b` và `prev_b<=nxt_b`
- Với NB = 2, cùng cạnh clock đó đảo `nxt_a`, `nxt_c`; đảo `nxt_bias` nếu `post`; đảo `nxt_b` nếu `reuse` = 0. Với NB = 1 mọi bank là 0

3.3.2. Bộ đếm chiếm dụng

Bảng 8: Bộ đếm của `gemm_bank_sb`

Bộ đếm	Rộng	Tăng khi	Giảm khi
`rcnt[A][b]`	2	Engine Đọc bắt đầu LOAD_A vào bank b	COMPUTE của job có `ba` = b xong
`rcnt[B][b]`	2	LOAD_B vào bank b, hoặc job `reuse` = 1 có `bb` = b đi qua bước B	COMPUTE của job có `bb` = b xong
`rcnt[BIAS][b]`	2	LOAD_BIAS vào bank b	COMPUTE của job `post` = 1 có `bbias` = b xong
`wcnt[C][b]`	1	COMPUTE của job có `bc` = b bắt đầu	STORE_C của job đó xong OKAY

Điều kiện bank

- LOAD_X vào bank b chỉ được bắt đầu khi `rcnt[X][b]` = 0: mọi job đọc bank đó đã tính xong. Job `reuse` = 1 không chờ (dữ liệu B của bank đó vẫn còn, vì chưa có LOAD_B nào vào bank ấy kể từ job trước)
- COMPUTE vào bank C b chỉ được bắt đầu khi `wcnt[C][b]` = 0: STORE_C cuối cùng đọc bank đó đã xong
- Bộ đếm không bao giờ tràn hay âm (assertion). `rcnt` tối đa 3 vì tối đa ba job đã gán bank mà chưa tính xong (engine Đọc, slot, engine Tính)
- Khi tác vụ kết thúc do lỗi, bộ đếm có thể khác 0; chúng được xóa khi chấp nhận tác vụ kế tiếp hoặc khi `soft_clr`

3.3.3. Cấp quyền bắt đầu pha

Mỗi engine tính tín hiệu `want` từ trạng thái đầu chu kỳ: `want_s` (Ghi), `want_c` (Tính), `want_r` (Đọc: FETCH hoặc một LOAD). `gemm_fsm_ctrl` trả `go`.

$$
\begin{aligned}
&\text{PP } (NB=2): && go_x = want_x \wedge \neg inh \\
&\text{SEQ } (NB=1): && go_s = want_s \wedge \neg act \wedge \neg inh, \quad go_c = want_c \wedge \neg act \wedge \neg want_s \wedge \neg inh, \\
& && go_r = want_r \wedge \neg act \wedge \neg want_s \wedge \neg want_c \wedge \neg inh
\end{aligned}
$$

Trong đó `act` = (engine Đọc đang chờ một lệnh DMA) $\vee$ (engine Tính ở RUN) $\vee$ (engine Ghi ở RUN); `inh` = tác vụ không ở trạng thái RUN (gồm SOFT, CLR). Ưu tiên Ghi > Tính > Đọc làm job cũ ra trước và bảo đảm lịch tuần tự.

Chặn sau lỗi

- `gemm_fsm_ctrl` cấp `blk_v`, `blk_idx` tổ hợp: bản ghi lỗi hiện có, gộp với lỗi đến trong chính chu kỳ này (`i_rd_error`, `i_wr_error`, kết quả CHECK sai), lấy chỉ số nhỏ nhất
- Mọi `want` của job có `idx` $\geq$ `blk_idx` bằng 0 khi `blk_v` = 1. Nhờ gộp lỗi cùng chu kỳ, không pha nào của descriptor $\geq$ f bắt đầu, kể cả ở đúng chu kỳ lỗi được báo
- Pha đã bắt đầu thì chạy tới hết (không abort khi lỗi). Kết quả của chúng không được ghi ra DRAM vì STORE_C của job $\geq$ f không bao giờ bắt đầu

3.4. Engine

3.4.1. Lệnh DMA

Bảng 9: Tham số lệnh DMA của mỗi pha

Pha	dst / fmt	base	rows	rwords	stride	bank
FETCH	3 (desc)	con trỏ	1	16	64	0
LOAD_BIAS	2 (BIAS)	`bias`	1	N	4N	`bbias`

Bảng 9: Tham số lệnh DMA của mỗi pha (tiếp theo)

Pha	dst / fmt	base	rows	rwords	stride	bank
LOAD_A	0 (A)	a	M	K/4	lda	ba
LOAD_B	1 (B)	b	K	N/4	ldb	bb
STORE_C	post	c	M	post ? N/4 : N	ldc	bc

3.4.2. Engine Đọc

Bảng 10: Trạng thái của gemm_rd_engine

Trạng thái	Làm gì	Sang
R_IDLE	Chờ	R_START khi ctrl khởi động tác vụ
R_START	Chế độ đơn: nạp job từ bản chụp. Chuỗi: con trỏ ← DESC_PTR, idx ← 0	R_CHECK (đơn) / R_FETCH (chuỗi)
R_FETCH	want_r = 1. Khi go_r: phát FETCH	R_FWAIT
R_FWAIT	Nhận word descriptor; chờ i_rd_done / i_rd_error	R_CHECK / R_STOP (lỗi mã 4)
R_CHECK	1 chu kỳ: gemm_job_check; nếu hợp lệ, gán bank	R_WAITP / R_STOP (lỗi mã 1)
R_WAITP	Nếu waitp = 1: chờ desc_done_cnt==idx	R_LBIAS nếu post, ngược lại R_LA
R_LBIAS, R_LA, R_LB	want_r = (rcnt[X][bank] = 0). Khi go_r: rcnt[X][bank]++, phát lệnh theo bảng Tham số lệnh DMA của mỗi pha	R_LWAIT
R_LWAIT	Chờ i_rd_done / i_rd_error	tiếp theo thứ tự BIAS → A → (B hoặc RB) → PUSH / R_STOP (lỗi mã 2)
R_RB	Job reuse: rcnt[B][bb]++, không phát lệnh	R_PUSH
R_PUSH	Khi slot trống: slot ← job. Chuỗi và last = 0: prev_kn ← (K, N), con trỏ ← next, idx++	R_FETCH; hoặc R_STOP nếu last = 1 hoặc chế độ đơn
R_STOP	Không làm gì tới hết tác vụ	—

Engine Đọc

- Ở mọi trạng thái chờ (R_FETCH, R_WAITP, R_LBIAS, R_LA, R_LB, R_PUSH): nếu job bị chặn thì sang R_STOP, không đẩy job vào slot
- Engine Đọc làm xong mọi LOAD của job d rồi mới FETCH job d+1 . WAIT_PREV không chặn FETCH và CHECK
- Bộ đệm descriptor ghi word i_desc_idx khi i_desc_valid. Dữ liệu của FETCH bị lỗi không được dùng
- o_perf_load = 1 khi ở R_LWAIT (không tính FETCH)

3.4.3. Engine Tính và engine Ghi

Bảng 11: Trạng thái của gemm_cp_engine và gemm_wr_engine

Trạng thái	Làm gì	Sang
C_IDLE	want_c = slot có job ∧ wcnt[C][slot.bc] = 0. Khi go_c: job ← slot, slot trống, wcnt++, o_pe_start = 1 ở chu kỳ kế tiếp	C_RUN
C_RUN	o_comp_active = o_perf_comp = 1. Khi i_pe_done: giảm rcnt[A][ba], rcnt[B][bb], và rcnt[BIAS][bbias] nếu post	C_HAND
C_HAND	Giữ job (và bank C) cho engine Ghi	C_IDLE khi engine Ghi nhận job
S_IDLE	want_s = engine Tính ở C_HAND. Khi go_s: job ← job của engine Tính, phát STORE_C ở chu kỳ kế tiếp	S_RUN
S_RUN	o_perf_store = 1. i_wr_done: wcnt[C][bc]--; ở chế độ chuỗi desc_done_cnt++; nếu last = 1, báo job cuối xong. i_wr_error: lỗi mã 3	S_IDLE

VÌ SAO ENGINE TÍNH GIỮ JOB Ở C_HAND KHÔNG LÀM CHẬM
COMPUTE(d+1) dùng bank C của job d−1, nên theo (COMPUTE(d) bắt đầu khi mọi LOAD của d và COMPUTE(d−1) đã xong, và STORE_C cuối cùng đọc bank C của d đã xong) chỉ bắt đầu được khi STORE_C(d−1) xong. Đúng lúc đó engine Ghi rảnh và nhận job d, nên engine Tính về C_IDLE và có thể bắt đầu d+1 ngay chu kỳ sau. Với NB = 1, lịch vốn đã tuần tự.

3.5. Điều khiển tác vụ

Bảng 12: Trạng thái của `gemm_fsm_ctrl`

Trạng thái	Làm gì	Sang
T_IDLE	Chờ. `i_start_pulse`: xóa bản ghi lỗi, `desc_done_cnt`, `err_desc_addr`, con trỏ bank, bộ đếm; xét `i_start_kind`	T_RUN. `i_soft_req`: T_CLR
T_RUN	Engine chạy. Kiểm điều kiện kết thúc	T_IDLE khi done/err; T_SOFT khi `i_soft_req`
T_SOFT	`inh` = 1; bật `o_rd_abort` khi kênh đọc bận, `o_wr_abort` khi kênh ghi bận; không báo done/err	T_CLR khi không engine nào chờ lệnh DMA
T_CLR	`o_soft_clr` = 1 trong 1 chu kỳ; mọi thanh ghi của FSM về giá trị reset	T_IDLE

Bảng 13: Xử lý `i_start_kind`

Giá trị	Hành vi
2'b01 (START)	Chế độ đơn: engine Đọc vào R_START
2'b10 (START_CHAIN)	`HAS_CHAIN` = 0: lỗi mã 5. `DESC_PTR`[5:0] $\neq$ 0: lỗi mã 1 . Ngược lại: chế độ chuỗi
2'b11	Lỗi mã 1 . Không engine nào chạy, không truy cập bus

Kết thúc tác vụ

- Bản ghi lỗi (`err_v`, `err_idx`, `err_code`, `err_addr`) chỉ được thay khi lỗi mới có idx nhỏ hơn . Mã: FETCH $\rightarrow$ 4, CHECK $\rightarrow$ 1, LOAD_* $\rightarrow$ 2, STORE_C $\rightarrow$ 3. `err_addr` = `daddr` của job lỗi; = con trỏ đang FETCH với lỗi mã 4; = `DESC_PTR` với (`DESC_PTR`[5:0] $\neq$ 0 được kiểm ở `gemm_fsm_ctrl` khi bắt đầu chuỗi: lỗi mã 1 với f = 0, `ERR_DESC_ADDR` = `DESC_PTR`, không truy cập bus)
- DONE: ở T_RUN, engine Ghi báo job có `last` = 1 xong OKAY và `err_v` = 0 $\rightarrow$ `o_done_pulse` ở chu kỳ kế tiếp
- ERROR: ở T_RUN, `err_v` = 1 và hệ yên: engine Đọc ở R_STOP và không chờ DMA; engine Tính không ở C_RUN; slot trống hoặc có idx $\geq$ f; engine Tính không giữ ở C_HAND một job idx $<$ f; engine Ghi ở S_IDLE $\rightarrow$ `o_err_pulse` với `o_err_code` = `err_code` ở chu kỳ kế tiếp; `o_err_desc_addr` $\leftarrow$ `err_addr` (chỉ ở chế độ chuỗi)
- Khi đó `o_desc_done_cnt` = f tự nhiên, vì mọi job $<$ f đã ghi xong và không job $\geq$ f nào được ghi
- `o_err_desc_addr` và `o_desc_done_cnt` bằng 0 ở chế độ đơn và khi `HAS_CHAIN` = 0

Soft reset

- `o_soft_busy` = 1 ở T_SOFT và T_CLR
- Ở T_SOFT: không pha mới nào bắt đầu; lệnh DMA đang chạy kết thúc bằng done hoặc error như thường ; engine vẫn nhận các xung đó để rời trạng thái chờ, nhưng không cập nhật bản ghi lỗi hay báo done
- COMPUTE đang chạy không được chờ: `o_soft_clr` hủy array và VPU
- `i_soft_req` ở T_IDLE: T_CLR ở chu kỳ kế tiếp. `i_soft_req` ở T_SOFT hoặc T_CLR: bỏ qua
- Nếu `i_soft_req` đến cùng chu kỳ với `i_rd_done` hoặc `i_wr_done`, kênh đó coi như đã rảnh

3.6. Timing

Bảng 14: Độ trễ cố định của FSM

Sự kiện	**Chu kỳ**
`i_start_pulse` → engine Đọc ở R_START	+1
R_CHECK	1 chu kỳ
`go_*` = 1 → `o_rd_start` / `o_wr_start` / `o_pe_start` = 1	+1
`i_*_done` → trạng thái kế tiếp của engine	+1
Điều kiện DONE / ERROR → `o_done_pulse` / `o_err_pulse`	+1
`i_soft_req` ở T_IDLE → `o_soft_clr`	+1

3.7. Reset, hiện thực, kiểm chứng

Reset

- `rst_n` = 0 hoặc T_CLR: mọi engine về trạng thái IDLE, slot trống, bản ghi lỗi, con trỏ bank, bộ đếm, `desc_done_cnt`, `err_desc_addr`, mọi xung đầu ra về 0. Trường dữ liệu của job không bắt buộc reset

Hiện thực

- SystemVerilog tổng hợp được; FSM viết theo kiểu hai khối (`always_ff` trạng thái, `always_comb` trạng thái kế); không có latch
- `job_t` khai báo trong `gemm_pkg` là `structpacked`; công cụ tổng hợp tự bỏ trường không dùng ở mỗi engine
- Với `HAS_CHAIN` = 0, logic FETCH, bộ đệm và kiểm tra descriptor, cùng `desc_done_cnt` được loại bằng `generate`. Với NB = 1, bộ đếm và con trỏ bank được loại, các điều kiện bank luôn đúng trừ quy tắc SEQ
- Mọi so sánh idx dùng 32 bit không dấu

Assertion bắt buộc (SVA)

- `o_rd_start` $\rightarrow$ kênh đọc rảnh; `o_wr_start` $\rightarrow$ kênh ghi rảnh; `o_pe_start` $\rightarrow$ `!i_pe_busy`
- NB = 1: tại mọi chu kỳ có tối đa một trong {engine Đọc chờ DMA, C_RUN, S_RUN}
- Không bộ đếm nào tràn hay âm
- LOAD_X bắt đầu vào bank b $\rightarrow$ không job nào ở slot, C_RUN hay C_HAND chưa tính xong có bank X = b (trừ job reuse)
- STORE_C bắt đầu theo thứ tự idx tăng dần 1
- `o_done_pulse` $\wedge$ `o_err_pulse` không bao giờ cùng lúc; mỗi `i_start_pulse` có tối đa một trong hai
- `o_comp_*` ổn định trong khi C_RUN

> **KIỂM CHỨNG BẰNG MÔ HÌNH**
> `gemm_sched_model.py` chạy 1 200 chuỗi ngẫu nhiên 1 – 7 descriptor (stride, `REUSE_B`, `WAIT_PREV` với descriptor đọc kết quả của descriptor trước, descriptor sai, lỗi AXI tiêm ngẫu nhiên ở FETCH/LOAD/STORE) với cả NB = 1 và NB = 2, và 300 tác vụ chế độ đơn. Mọi bên đọc lấy dữ liệu ở đầu và cuối pha và phải thấy giống nhau, nên ghi đè sai bank bị phát hiện. Kết quả: DRAM khớp thực thi tuần tự ở mọi chuỗi DONE; với lỗi, `DESC_DONE_CNT` = f, `ERR_DESC_ADDR` đúng, không pha nào của job $\geq$ f bắt đầu sau lỗi; NB = 1 không có hai pha chồng nhau. Mô hình đã bắt hai lỗi quy tắc trong quá trình thiết kế (đường `REUSE_B` và lỗi cùng chu kỳ), dẫn tới bước R_RB và (`gemm_fsm_ctrl` cấp `blk_v`, `blk_idx` tổ hợp: bản ghi lỗi hiện có, gộp với lỗi đến trong chính chu kỳ này (`i_rd_error`, `i_wr_error`, kết quả CHECK sai), lấy chỉ số nhỏ nhất).